Contador de Objetos por Color en Video

Guía de Instalación y Uso

`OpenCV 4.6 + Rust 1.94 + yt-dlp`

Sistema Operativo Compatible
Pop!OS / Ubuntu 22.04+

Tecnología	Versión
Rust	1.94.1+
OpenCV	4.6.0
Crate opencv	0.93.7
Python / yt-dlp	3.12 / último

Sistemas Embebidos — 30 de marzo de 2026

Índice

1. Descripción del Proyecto **2**
1.1. Colores Detectados . . . 2

2. Requisitos del Sistema **2**
2.1. Requisitos de Hardware . . . 2
2.2. Requisitos de Software . . . 3

3. Instalación Paso a Paso **3**
3.1. Paso 1: Verificar o Instalar Rust . . . 3
3.2. Paso 2: Instalar Dependencias del Sistema . . . 3
3.3. Paso 3: Instalar Dependencias de Python . . . 4
3.4. Paso 4: Crear la Estructura del Proyecto . . . 4
3.5. Paso 5: Configurar el Cargo.toml . . . 4
3.6. Paso 6: Copiar el Código Fuente . . . 4
3.7. Paso 7: Compilar el Proyecto . . . 5

4. Uso del Sistema **5**
4.1. Sintaxis General . . . 5
4.2. Opciones Disponibles . . . 5
4.3. Ejemplos de Uso . . . 5
4.3.1. Video Local . . . 5
4.3.2. Video de YouTube . . . 5
4.3.3. Con Opciones Avanzadas . . . 6
4.4. Controles Durante la Ejecución . . . 6

5. Solución de Problemas Comunes **6**
5.1. Error: `fatal error: 'memory' file not found` . . . 6
5.2. Error: `numpy.dtype size changed` . . . 6
5.3. Error: `No module named 'lap'` . . . 7
5.4. El video va muy rápido . . . 7
5.5. El video de YouTube no está disponible . . . 7

6. Estructura del Proyecto **7**

7. Reporte Final **7**

1. Descripción del Proyecto

Este proyecto implementa un sistema de **detección y conteo de objetos por color** en video, desarrollado en **Rust** utilizando la librería **OpenCV**. El sistema es capaz de:

- Procesar videos locales en formato MP4 u otros formatos compatibles.
- Obtener streams en tiempo real desde **YouTube** mediante `yt-dlp`.
- Detectar y segmentar objetos según su color dominante usando **filtros HSV**.
- Mostrar estadísticas en tiempo real con un dashboard visual.
- Generar un reporte final en consola con porcentajes por color.

1.1. Colores Detectados

El sistema detecta los siguientes 8 colores:

#	Color	Rango H (HSV)	Aplicación típica
1	Rojo	0–10° y 160–180°	Señales, ropa roja
2	Naranja	10–25°	Taxis, conos
3	Amarillo	25–35°	Chalecos, frutas
4	Verde	35–85°	Vegetación, ropa
5	Azul	100–130°	Ropa, cielo, agua
6	Violeta	130–160°	Ropa
7	Blanco	S <30, V >200	Ropa blanca
8	Negro	V <40	Ropa oscura

2. Requisitos del Sistema

Nota

Esta guía está orientada a sistemas **Pop!OS** y **Ubuntu 22.04 o superior**. Los comandos pueden variar en otras distribuciones.

2.1. Requisitos de Hardware

- Procesador de 64 bits (x86_64)
- Mínimo 4 GB de RAM (recomendado 8 GB)
- Conexión a internet (para descarga de dependencias y videos de YouTube)
- Espacio en disco: mínimo 2 GB libres

2.2. Requisitos de Software

- Pop!OS 22.04+ / Ubuntu 22.04+
- Rust 1.70 o superior
- Python 3.10 o superior
- OpenCV 4.x instalado en el sistema

3. Instalación Paso a Paso

3.1. Paso 1: Verificar o Instalar Rust

Verificar si Rust ya está instalado:

```
rustc --version
cargo --version
```

Si no está instalado, ejecutar:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

Consejo

Cerrar y volver a abrir la terminal después de instalar Rust para que los cambios surtan efecto.

3.2. Paso 2: Instalar Dependencias del Sistema

Ejecutar el siguiente comando para instalar todas las dependencias necesarias:

```
sudo apt install libopencv-dev clang libclang-dev \
    build-essential libstdc++-14-dev g++ python3-pip -y
```

Paquete	Para qué sirve
`libopencv-dev`	Headers y librerías de OpenCV
`clang / libclang`	Compilador C++ requerido por el crate opencv
`build-essential`	Herramientas de compilación base
`libstdc++-14-dev`	Headers de C++ estándar
`g++`	Compilador C++
`python3-pip`	Gestor de paquetes Python

3.3. Paso 3: Instalar Dependencias de Python

```
pip3 install yt-dlp --break-system-packages
pip3 install "numpy<2.0" --break-system-packages --force-
    reinstall
pip3 install lapx --break-system-packages
```

Advertencia

El flag `-break-system-packages` es necesario en Pop!OS y Ubuntu 22.04+ debido a restricciones en la gestión de paquetes Python del sistema.

3.4. Paso 4: Crear la Estructura del Proyecto

```
mkdir -p ~/proyecto_openCV/src
cd ~/proyecto_openCV
```

3.5. Paso 5: Configurar el Cargo.toml

Crear y editar el archivo de configuración:

```
nano Cargo.toml
```

Pegar el siguiente contenido exactamente:

Listing 1: Cargo.toml

```
[package]
name = "proyecto_openCV"
version = "0.1.0"
edition = "2021"

[dependencies]
opencv = { version = "0.93", features = ["videoio", "imgproc", "
    highgui"] }
```

Guardar con `Ctrl+O` y salir con `Ctrl+X`.

Advertencia

Usar `edition = "2021"` y **no** `edition = "2024"`, ya que el crate de opencv no es compatible con la edición 2024 actualmente.

3.6. Paso 6: Copiar el Código Fuente

```
nano src/main.rs
```

Pegar el contenido completo del archivo `main.rs` provisto. Guardar con `Ctrl+O` y salir con `Ctrl+X`.

3.7. Paso 7: Compilar el Proyecto

```
cargo build --release
```

Advertencia

La **primera compilación** puede tardar entre 5 y 15 minutos porque Rust descarga y compila todas las dependencias desde cero. Las compilaciones posteriores serán casi inmediatas.

Si la compilación es exitosa, se verá:

```
Finished `release` profile [optimized] target(s) in X.XXs
```

4. Uso del Sistema

4.1. Sintaxis General

```
cargo run --release -- [OPCIONES]
```

4.2. Opciones Disponibles

Opción	Valor	Descripción
`-video`	ruta/archivo.mp4	Usar un video local
`-youtube`	URL	Usar video de YouTube
`-skip`	N (default: 3)	Procesar 1 de cada N frames
`-min-area`	px (default: 500)	Área mínima de objeto detectado

4.3. Ejemplos de Uso

4.3.1. Video Local

```
cargo run --release -- --video "/ruta/al/video.mp4"
```

4.3.2. Video de YouTube

```
cargo run --release -- --youtube "https://www.youtube.com/watch?
    v=VIDEO_ID"
```

4.3.3. Con Opciones Avanzadas

```
# Procesar cada frame (mas lento, mas preciso)
cargo run --release -- --video "video.mp4" --skip 1

# Objetos mas grandes solamente
cargo run --release -- --video "video.mp4" --min-area 1500

# Combinacion de opciones
cargo run --release -- \
    --youtube "https://youtu.be/XXXX" \
    --skip 2 \
    --min-area 800
```

4.4. Controles Durante la Ejecución

Tecla	Acción
`q`	Salir y mostrar reporte final
`Esc`	Salir también

5. Solución de Problemas Comunes

5.1. Error: `fatal error: 'memory' file not found`

Causa: Faltan los headers de C++ estándar.

Solución:

```
sudo apt install libstdc++-14-dev g++ -y
cargo clean
cargo build --release
```

5.2. Error: `numpy.dtype size changed`

Causa: Conflicto entre versiones de numpy del sistema y pip.

Solución:

```
pip3 install "numpy<2.0" --break-system-packages --force-
   reinstall
```

5.3. Error: `No module named 'lap'`

Causa: Falta la librería de asignación lineal para tracking.

Solución:

```
pip3 install lapx --break-system-packages
```

5.4. El video va muy rápido

Causa: Rust procesa los frames más rápido que el FPS original.

Solución: Usar `-skip 1` para que el delay se calcule correctamente con el FPS real del video:

```
cargo run --release -- --video "video.mp4" --skip 1
```

5.5. El video de YouTube no está disponible

Causa: Restricción geográfica o video privado.

Solución: Descargar el video con yt-dlp y usarlo como video local:

```
yt-dlp "URL_DEL_VIDEO" -o "video.mp4"
cargo run --release -- --video "video.mp4"
```

6. Estructura del Proyecto

```
proyecto_openCV/
    |-- Cargo.toml        <- Configuracion y dependencias
    |-- Cargo.lock        <- Versiones exactas (generado auto)
    |-- src/
    |   `-- main.rs       <- Codigo fuente principal
    `-- target/
        `-- release/
            `-- proyecto_openCV  <- Ejecutable compilado
```

7. Reporte Final

Al finalizar el video o presionar `q`, el sistema imprime en consola un reporte como el siguiente:

```
=====================================================
      REPORTE FINAL - CONTADOR DE COLORES
=====================================================
```

```
  Frames procesados : 450
  Tiempo total      : 38.2 segundos

  OBJETOS DETECTADOS POR COLOR:
  ----------------------------------------
  Azul          1823  detecciones                          38%
  Negro         1340  detecciones                     28%
  Blanco         960  detecciones                  20%
  Rojo           670  detecciones              14%

  COLOR MAS FRECUENTE: AZUL
=====================================================
```